

<Promesas>

<Componentes>



Desarrollo de aplicaciones web con JavaScript for Beginners

Let's start! →



<javascript>

<angular>



<https://github.com/JennyPereira/web-development-js-angular>

<HTML>

<TypeScript>

— ○ ×

Tabla de `Contenidos`

{01} Introducción

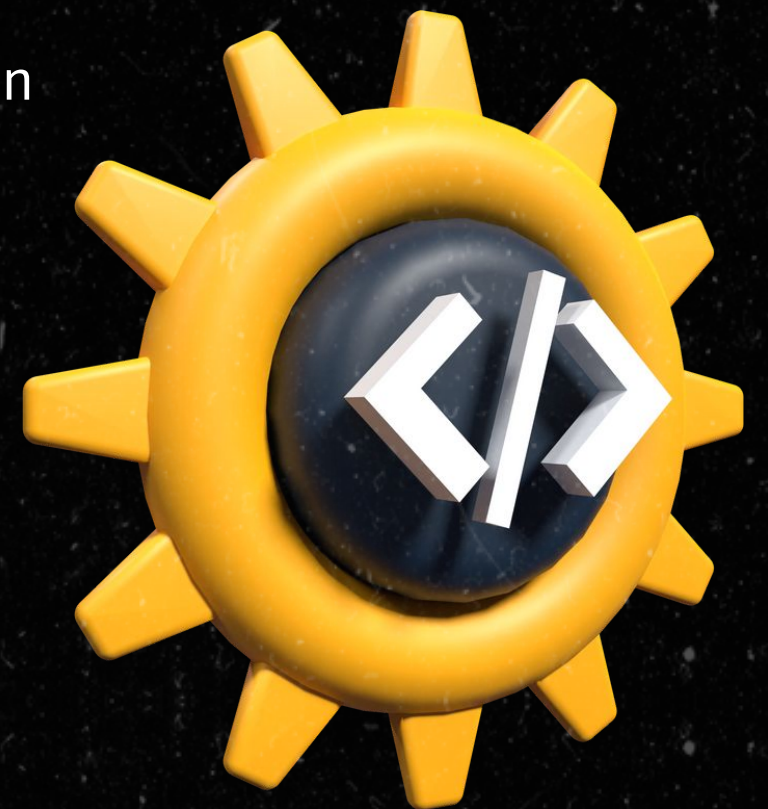
Conceptos previos e importantes de JavaScript

{02} Introducción a Angular

Elaboración de un ejemplo sencillo con Angular

{03} {Aplicación con Angular}

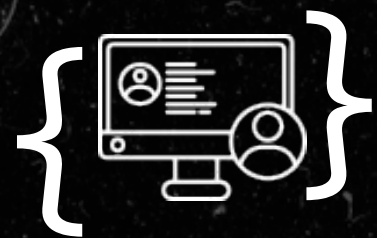
Se revisa temas avanzados en Angular (servicios, rutas, etc)



Tecnologías por /User'



- NodeJS - v20.15.1
- npm - 10.7.0
- json-server



Angular 18



{01} Tipos de variables

const

let

var

```
11  ✓ if (true) {  
12      let y = 20;  
13      console.log(y); // 20  
14  }  
15  console.log(y); // Error: y is not defined
```

```
11  ✓ function example() {  
12  ✓      if (true) {  
13          var x = 10;  
14      }  
15      console.log(x); // 10  
16  }
```

```
11  const z = 30;  
12  z = 40; // Error: Assignment to constant variable
```



{02} Funciones

Función simple

Expresiones function

Funciones flecha

Funciones callback

Manejo de errores

```
4 function firstProgram() {  
5     cadena = "hola mundo"  
6     return cadena;  
7 }
```

```
4 const firstProgram = function () {  
5     cadena = "hola mundo"  
6     return cadena;  
7 }
```

```
15 // Función tradicional  
16 function example(a) {  
17     return a + 100;  
18 }  
19  
20 // 1. Elimina la palabra "function" y coloca la flecha entre el argumento y el corchete de apertura.  
21 (a) => {  
22     return a + 100;  
23 }  
24  
25 // 2. Quita los corchetes del cuerpo y la palabra "return" – el return está implícito.  
26 (a) => a + 100;  
27  
28 // 3. Suprime los paréntesis de los argumentos  
29 a => a + 100;
```



{03} Más funciones

Funciones callback

Promesas

Manejo de errores

*Según developer mozilla,
es un objeto que
representa la
culminación o fracaso de
una operación asíncrona*

```
4 app.get("/", function (req, res) {  
5   |   caden = "hola Mundo"  
6   |   res.send(caden);  
7   | })
```

```
async getCoffeeProductsAvailable(): Promise<CoffeeProduct[]> {  
  const data = await fetch(this.url)  
    .then(response => response.json())  
    .then(info => {  
      const datos = info.filter((item: { available: boolean; }) => item.available == true);  
      return datos;  
    })  
    .catch((e) => {  
      console.log(e);  
    });  
  return await data ?? [];  
}
```



<DOM>

<html>



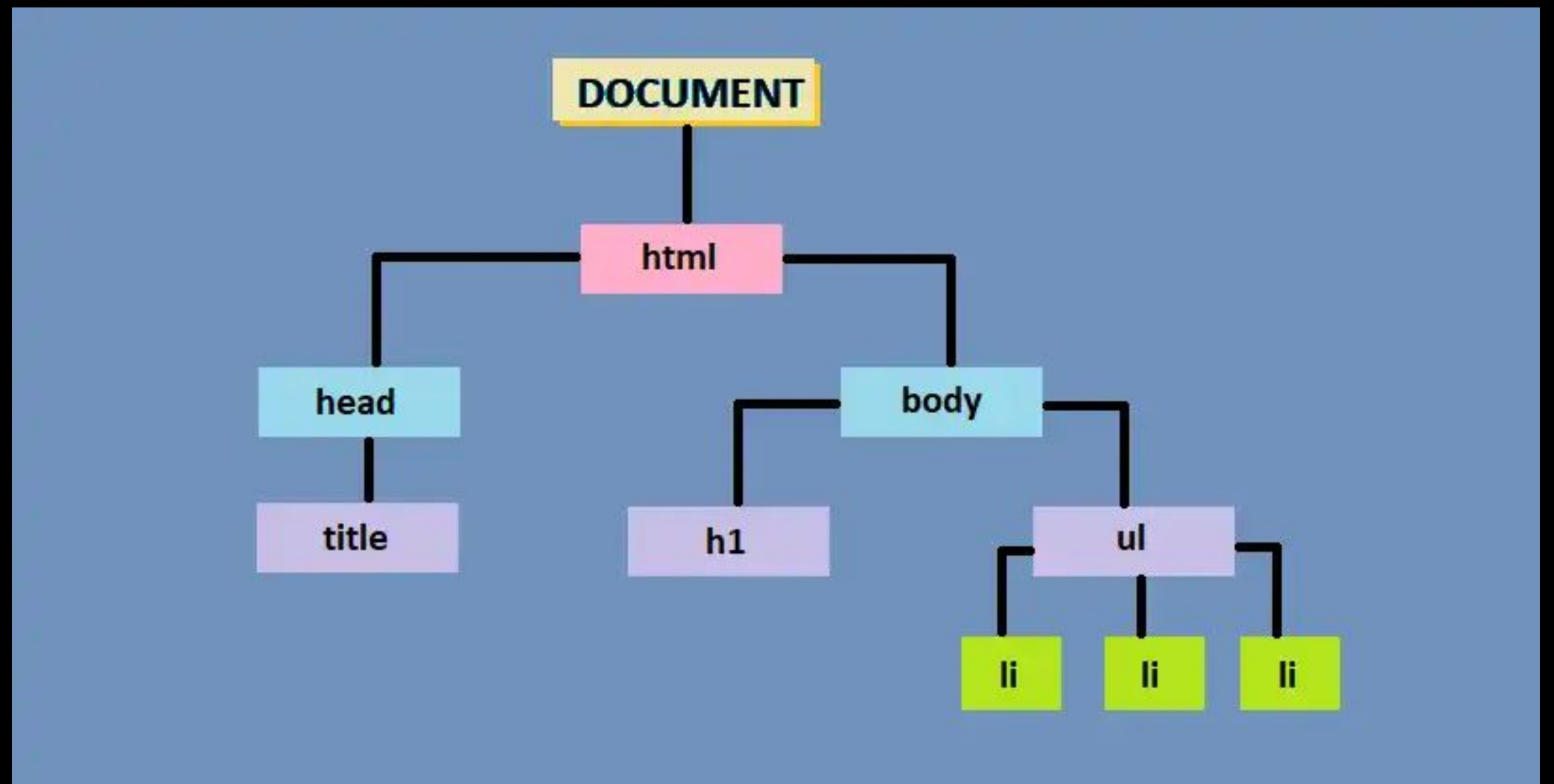
```
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Document</title>
8  </head>
9
10 <body>
11     <h1>Título de nivel 1</h1>
12     <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Atque paria
13         possimus inventore itaque! Quaerat ex laborum dolore expedita eum a
14     </p>
15 </body>
16
17 </html>
```



{04} DOM

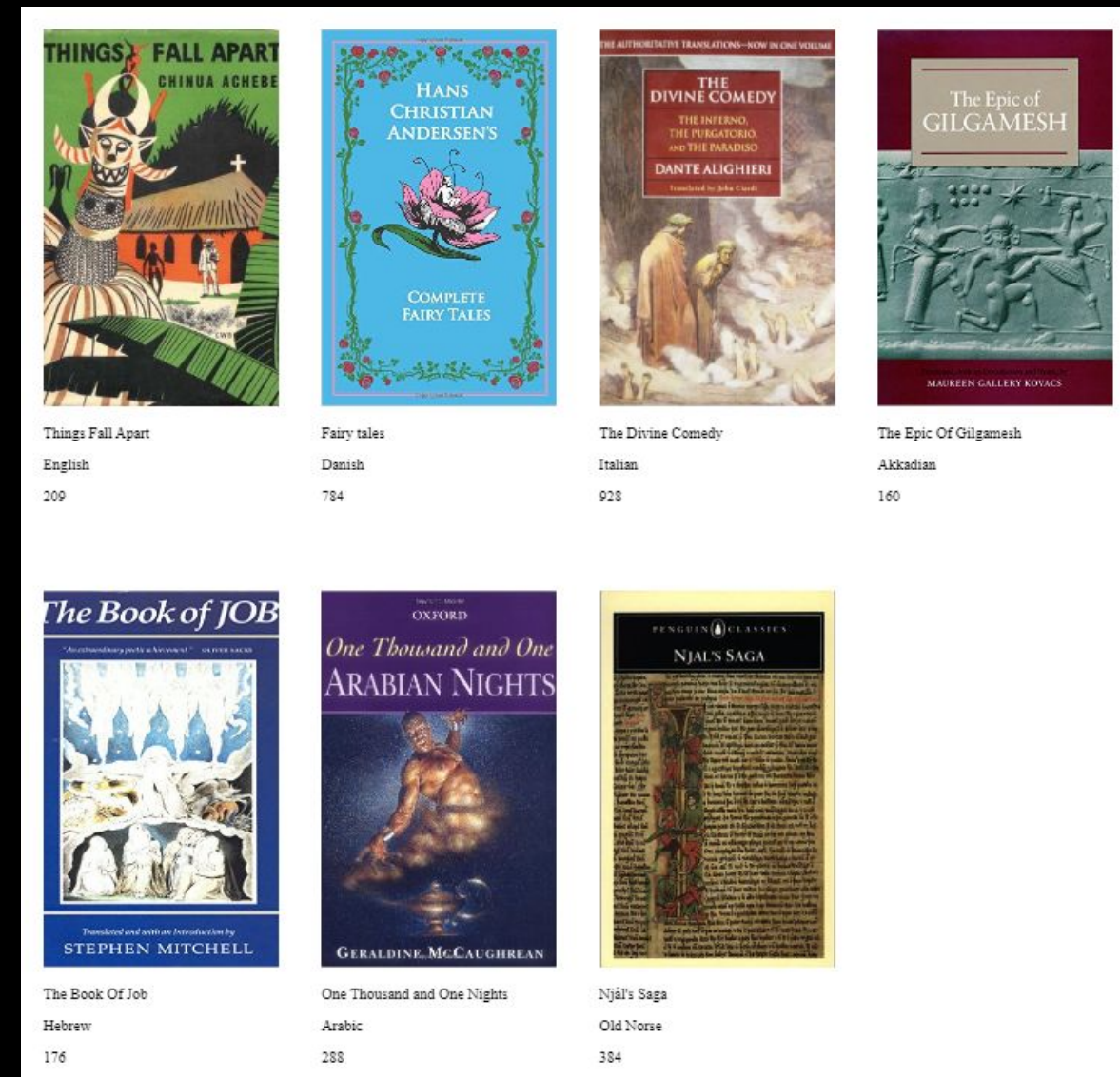
¿Qué es el DOM?

Es una representación
en memoria de un
documento HTML.



{05} Ejercicio DOM

- Descargar de GitHub
- Crear nuevos elementos
- Asignar clases
- Agregar al HTML
- Crear elementos hermanos



<Company>

<Lesson>



Repaso

- Visual studio Code
- CSS

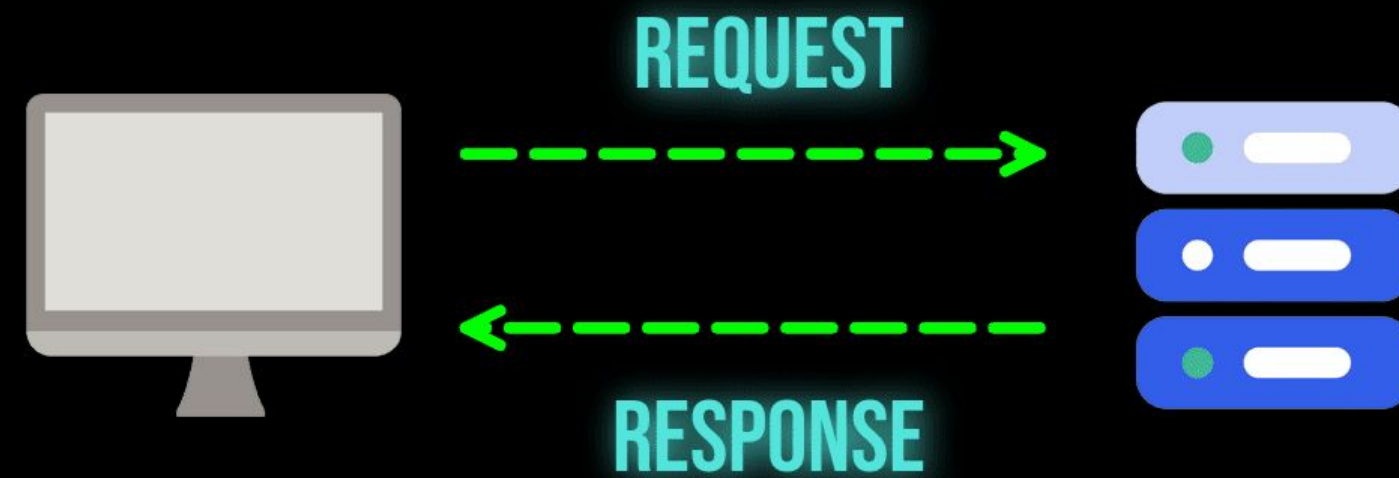


<Angular>

<Component>



Arquitectura cliente/servidor



<Angular>

<Component>

— ○ ✕

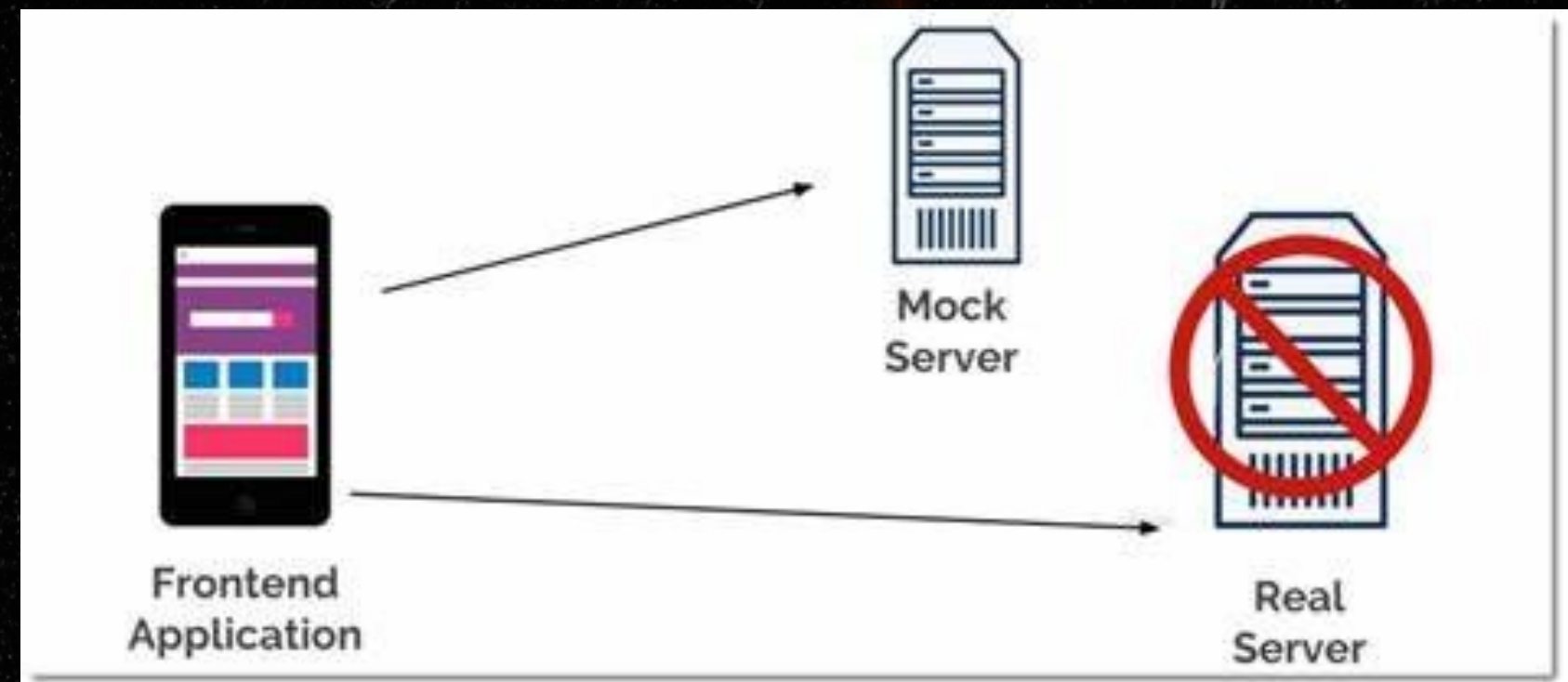
Estructura de un archivo JSON

```
data > {} peliculas.json > ...  
1  [  
2    {  
3      "id": 1,  
4      "title": "Comprar shampoo",  
5      "text": "Máximo hasta 20 febrero"  
6    },  
7    {  
8      "id": 2,  
9      "title": "Limpiar el refrigerador",  
10     "text": "Sacar todo tipo de comida perecible"  
11   },  
12   {  
13     "id": 3,  
14     "title": "Lavar el carro",  
15     "text": "Controlar e inspeccionar que los tres liquidos principales esten al día"  
16   }  
17 ]
```

{MOCKS}

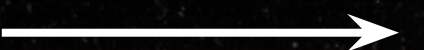
{} ¿Qué es?

Es una versión simulada de algo real que se utiliza para propósitos de prueba, revisión o demostración.



Crear mocks

1. Inicializar proyecto `npm init -y`
2. Instalar `npm install -g json-server`
3. Crear archivo `db.json`
4. Iniciar el servidor con `json-server --watch db.json`



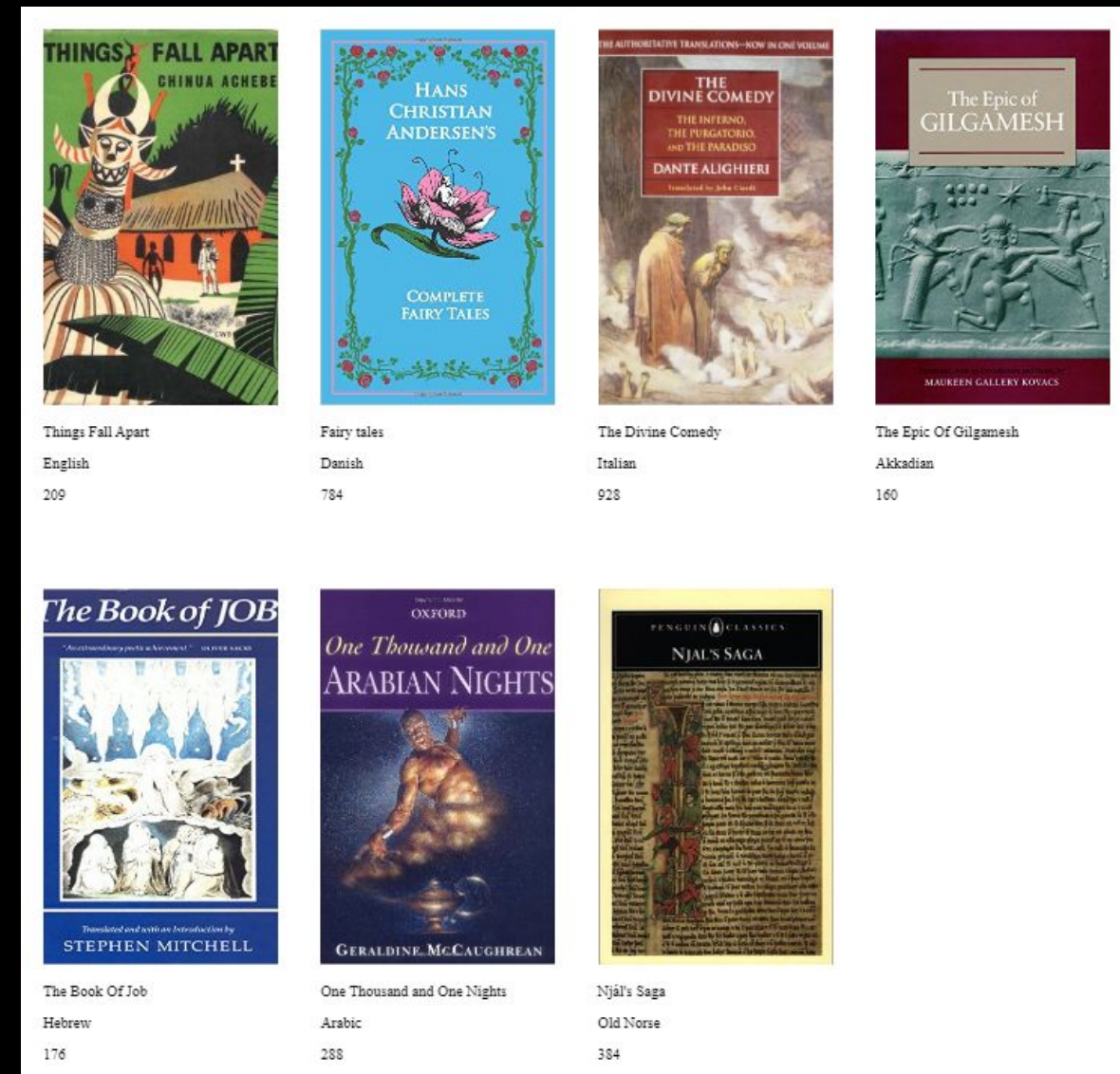
<Company>

<Lesson>



{03} Sollicitud HTTP

- Async/Await
- API Fetch
- Axios



<Angular>

<Typescript>



Angular

Single-page application (SPA). Es un sitio web o una aplicación web que reescribe dinámicamente una página web actual con nuevos datos del servidor web, en lugar del método predeterminado de un navegador web que carga páginas completamente nuevas.



<Company>

<Lesson>



{01} Versionamento



{02} Version 18

Lanzado en Mayo de 2024

1. Compiladores como Esbuild y Vite por defecto. Esto acelera el proceso de compilación
 2. Formularios: Eventos de Cambio de Estado y Valor
 3. SSR y SSG Mejorados: El renderizado del lado del servidor (SSR) y el renderizado estático del sitio (SSG) han sido optimizados
 4. Nuevas Directivas para Control de Flujos
 - i. Directiva @if
 - ii. Directiva @switch
- a. Es esencial tener versiones nodejs actualizadas



<Company>

<Lesson>



Instalación de Angular

Paso 01

Instalaremos Angular con el siguiente comando:

```
npm install -g @angular/cli
```

Paso 02

Comprobar versión:
`ng version`

Paso 3

Crear nuestro primer proyecto con Angular:

```
ng new simple-coffee-app
```



{03} Estructura de un proyecto Angular

1. package.json
2. main.ts
3. src/app
4. Estructura de un componente
 - a. Archivo .ts
 - b. Archivo .css
 - c. Archivo .html

```
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-coffe-sending',
5    standalone: true,
6    imports: [],
7    templateUrl: './coffe-sending.component.html',
8    styleUrls: ['./coffe-sending.component.scss']
9  })
10 export class CoffeSendingComponent {
11
12 }
```

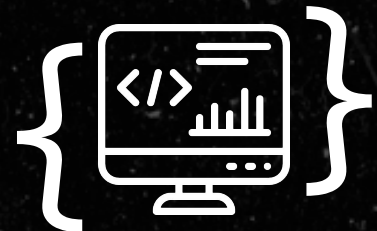


<Angular>

<typeScript>

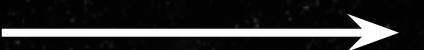


Creando nuestro primer componente



```
ng g c components/component-test
```

```
ng g c components/coffee-product-form --skip-tests
```



<Angular>

<typeScript>



DATABINDING

Template
.html file

String interpolation
{{ data }}

Property binding
[property] = "data"

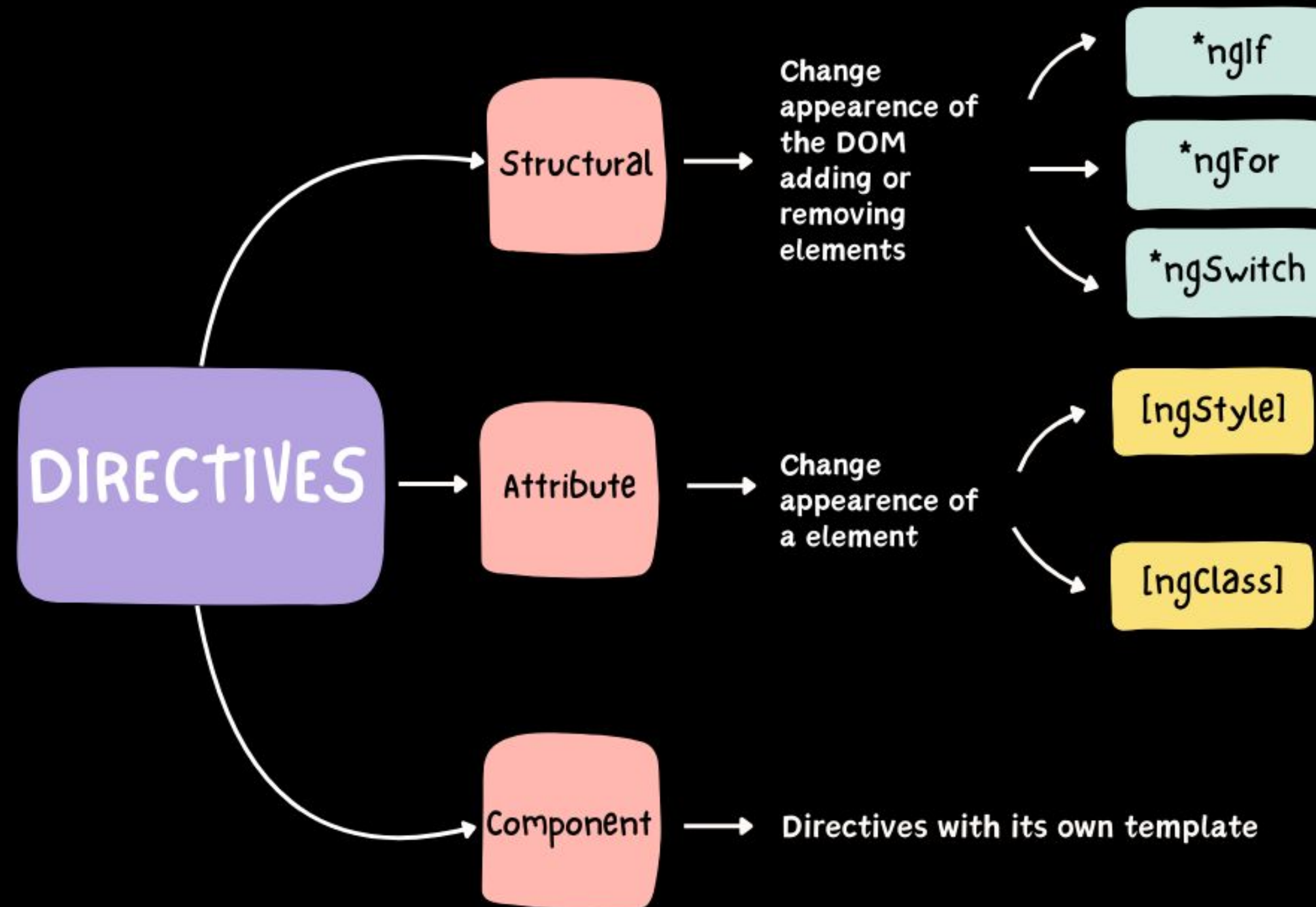
TypeScript
Code
.ts file

Event Binding
(event)="expression"

Two-Way-Binding
[(ngModel)] = "data"

<Angular>

<typeScript>



<Angular>

<typeScript>



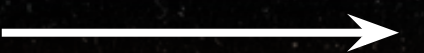
{}ngStyle

{}ngClass

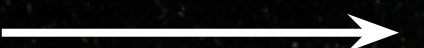
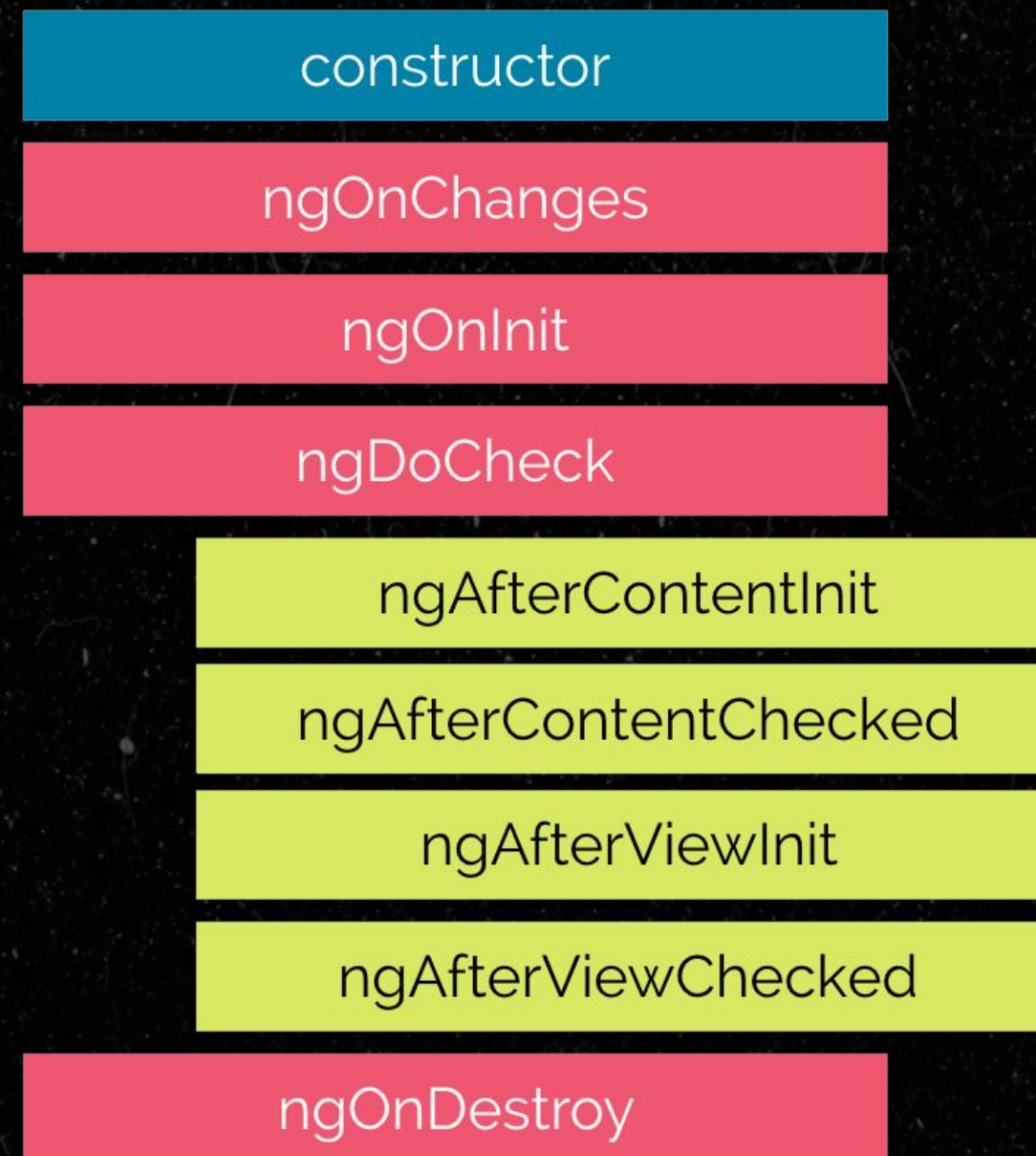
```
<div [ngStyle]="{'background-color':person.country === 'UK' ? 'green' : 'red' }"></div>
```

```
33 <ul *ngFor="let person of people">
34   <li [ngClass]="{
35     'text-success':person.country === 'UK',
36     'text-primary':person.country === 'USA',
37     'text-danger':person.country === 'HK'
38   }">{{ person.name }} ({{ person.country }})
39   </li>
40 </ul>
```

```
25 <ul *ngFor="let person of people">
26   <li [class.text-success]="person.country === 'UK'"
27     [class.text-primary]="person.country === 'USA'"
28     [class.text-danger]="person.country === 'HK'"
29     {{ person.name }} ({{ person.country }})
30   </li>
31 </ul>
```



{04} Ciclo de vida de un componente



<Angular>

<Typescript>



Coffee Store APP

<https://devchallenges.io/challenge/simple-coffee-listing>



<Angular>

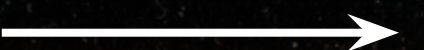
<Component>



{01} Crear Modelo

```
ng g i CoffeeProduct
```

```
export interface CoffeeProduct {  
  id: number;  
  name: string;  
  image: string;  
  price: string;  
  rating: number;  
  votes: number;  
  popular: boolean;  
  available: boolean;  
}
```



<Angular>

<Component>



{02} Crear servicio

```
ng g s coffe --skip-tests
```

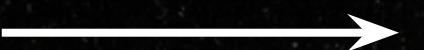
```
import { Injectable } from '@angular/core';
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class CoffeService {
```

```
  constructor() { }
```

```
}
```



<Angular>

<Component>



Comunicación entre componentes

Custom Event Binding & Component Property Binding

app.component.ts

```
import {Component} from '@angular/core';
import {Student} from './student';
@Component({
  selector: 'custom-event',
  templateUrl: 'app/app.component.html'
})
export class AppComponent {
  title = 'Update Person';
  fullName = '';
  studentObj = new Student(100, 'Mahesh', 'Sharma');
  saveData(emp) {
    this.fullName = emp.fname + ' ' + emp.lname;
  }
}
```

app.component.html

```
<h3>{{title}}</h3>
<person-app [updateEmployee]="saveData($event)"
  [student]="studentObj"> </person-app>
<p>{{studentObj.fname + ' ' + studentObj.lname}} </p>
<p>{{fullName}}</p>
```

Component Property Binding

Custom Event Binding

person.component.ts

```
import {Component, EventEmitter, Input, Output} from '@angular/core';
import {Student} from './student';
import {Employee} from './employee';
@Component({
  selector: 'person-app',
  templateUrl: 'app/person.component.html'
})
export class PersonComponent {
  @Input() student : Student;
  @Output() updateEmployee = new EventEmitter<Employee>();
  employee = new Employee();
  update() {
    this.employee.fname = this.student.fname;
    this.employee.lname = this.student.lname;
    this.updateEmployee.emit(this.employee);
  }
}
```

person.component.html

```
<div>
  <b>Id: {{student.id}} </b> <br/>
  <input [value]="student.fname"
    (input)="student.fname=$event.target.value" /> <br/>
  <input [value]="student.lname"
    (input)="student.lname=$event.target.value" /> <br/>
  <br/>
  <button (click)="update()">Update</button>
</div>
```

Property Binding

Event Binding

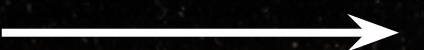
<Angular>

<Component>



{03} Rutas en angular

```
<nav>  
  <a routerLink="">Home</a>  
  <a routerLink="/newproduct">New product</a>  
</nav>
```



<Company>

<Lesson>



/Otras funciones con servicios'



Observable

Elaborate on what you
want to discuss.



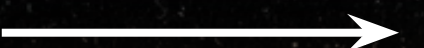
Pipe

Elaborate on what you
want to discuss.



Subscribe

Elaborate on what you
want to discuss.



<Company>

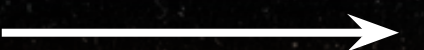
<Lesson>



Aplicación con Angular 'Coffee Store'

{} Componente Catálogo

{} Componente Producto



<Angular>

<Component>

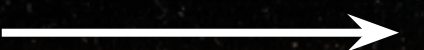


{01} Nuevo Proyecto

```
ng new coffee-store --strict
```

```
ng g c components/coffeeCatalog --skip-tests
```

Crear el componente coffeeProduct



<Angular>

<Component>



{04} Crear servicio

```
ng g s coffe --skip-tests
```

```
import { Injectable } from '@angular/core';
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class CoffeService {
```

```
  constructor() { }
```

```
}
```

